

SymPy Bug Report

1 Bug 48: Core algebraic identities erase incomplete Piecewise undefined branches

1.1 Minimal Reproducer

```
from sympy import Eq, Piecewise, symbols

x = symbols("x", real=True)
p = Piecewise((1, x > 0))

print("p.subs(x, -1) =", p.subs(x, -1))
print("(p*0).subs(x, -1) =", (p * 0).subs(x, -1))
print("(p-p).subs(x, -1) =", (p - p).subs(x, -1))
print("Eq(p, p).subs(x, -1) =", Eq(p, p).subs(x, -1))
```

1.2 Observed Behavior

```
p.subs(x, -1) = nan
(p*0).subs(x, -1) = 0
(p-p).subs(x, -1) = 0
Eq(p, p).subs(x, -1) = True
```

SymPy correctly exposes the missing branch of the original incomplete `Piecewise` as `nan` at $x = -1$. However, after applying core algebraic identities, the same undefined point is converted into total finite information: 0, 0, and True.

1.3 Expected Behavior

The undefined behavior of the incomplete `Piecewise` should be preserved, or the simplifying identities should be withheld where they would erase the implicit undefined region. In particular, at $x = -1$, $(p \cdot 0)$ and $p - p$ should still evaluate to NaN, and $\text{Eq}(p, p)$ should not evaluate to unconditional truth.

1.4 Mathematical Explanation

The expression

$$p(x) = \begin{cases} 1, & x > 0 \end{cases}$$

has no branch for $x \leq 0$. SymPy's own evaluation semantics therefore make $p(-1)$ undefined, represented by NaN.

The identities $z \cdot 0 = 0$, $z - z = 0$, and $z = z$ are valid for ordinary finite values z , but they are not pointwise valid when z is undefined. At the test point, the relevant value is $z = p(-1) = \text{NaN}$. An undefined value multiplied by zero is still undefined, subtracting an undefined value from itself is still undefined, and equality of two undefined evaluations is not a proof of pointwise truth. Thus the returned results are not merely alternate representations of the same partial function; they extend the domain and assert concrete values on a point where the original expression has no finite value.

1.5 Source-Level Diagnosis

The diagnosis identifies the responsible behavior in SymPy’s generic core simplification layer. The incomplete `Piecewise` is constructed in `sympy/functions/elementary/piecewise.py`; because it has no final `True` branch, it remains partial, and the missing-branch behavior is documented as evaluation to `NaN`. The traced call path then sends `p*0` through multiplication simplification in `sympy/core/mul.py`, `p-p` through term combination in `sympy/core/add.py` and `sympy/core/mul.py`, and `Eq(p, p)` through `Equality.__new__` in `sympy/core/relational.py`.

For multiplication, the diagnosis points especially to `sympy/core/mul.py` around lines 529–535, where collected factors with zero exponent are discarded according to the ordinary rule $x^0 \mapsto 1$ and related canonicalization can collapse a product with numeric zero to the scalar 0. For equality, it points to `sympy/core/relational.py` around lines 625–634, where structurally identical operands are evaluated through `is_eq` and become `True`. These rules are sound for total finite expressions, but they do not check whether an operand may be partial because it contains an incomplete `Piecewise`.

Consequently, the simplifier removes the expression whose evaluation would have reached the implicit `NaN` branch. Substitution after simplification therefore sees only a scalar 0 or unconditional truth, rather than the original partial expression. A plausible fix direction, supported by the diagnosis, is for core identity simplifications that collapse expressions containing incomplete `Piecewise` objects to preserve the undefined condition explicitly, for example as a `Piecewise`, or to decline the simplification when the operand can evaluate to `NaN` on part of its domain.

1.6 Additional Failing Instances

The independent verification covers the family

$$p_a(x) = \begin{cases} a + 1, & x > a \end{cases} \quad \text{evaluated at} \quad x = a - 1,$$

for $a = 0, 1, 2, 3, 4, 5$. In every case, $a - 1$ lies outside the covered branch, so the original expression is undefined there. The expected behavior is the same across the family because the algebraic identities are being applied to an undefined value, not to an ordinary finite value.

Input / Expression	SymPy behavior	Expected behavior
$a = 0$: $p = \text{Piecewise}((1, x > 0))$, evaluate at $x = -1$	$p = \text{NaN}$, but $p \cdot 0 = 0$, $p - p = 0$, $\text{Eq}(p, p) = \text{True}$	$p \cdot 0 = \text{NaN}$, $p - p = \text{NaN}$, equality not True
$a = 1$: $p = \text{Piecewise}((2, x > 1))$, evaluate at $x = 0$	$p = \text{NaN}$, but $p \cdot 0 = 0$, $p - p = 0$, $\text{Eq}(p, p) = \text{True}$	$p \cdot 0 = \text{NaN}$, $p - p = \text{NaN}$, equality not True
$a = 2$: $p = \text{Piecewise}((3, x > 2))$, evaluate at $x = 1$	$p = \text{NaN}$, but $p \cdot 0 = 0$, $p - p = 0$, $\text{Eq}(p, p) = \text{True}$	$p \cdot 0 = \text{NaN}$, $p - p = \text{NaN}$, equality not True
$a = 3$: $p = \text{Piecewise}((4, x > 3))$, evaluate at $x = 2$	$p = \text{NaN}$, but $p \cdot 0 = 0$, $p - p = 0$, $\text{Eq}(p, p) = \text{True}$	$p \cdot 0 = \text{NaN}$, $p - p = \text{NaN}$, equality not True
$a = 4$: $p = \text{Piecewise}((5, x > 4))$, evaluate at $x = 3$	$p = \text{NaN}$, but $p \cdot 0 = 0$, $p - p = 0$, $\text{Eq}(p, p) = \text{True}$	$p \cdot 0 = \text{NaN}$, $p - p = \text{NaN}$, equality not True
$a = 5$: $p = \text{Piecewise}((6, x > 5))$, evaluate at $x = 4$	$p = \text{NaN}$, but $p \cdot 0 = 0$, $p - p = 0$, $\text{Eq}(p, p) = \text{True}$	$p \cdot 0 = \text{NaN}$, $p - p = \text{NaN}$, equality not True

1.7 Related Issues Assessment

The available deduplication evidence lists PR #29518 and issues #21385 and #17438 as related public material. The artifact does not identify a clear public duplicate of this exact incomplete-**Piecewise** counterexample; instead, its verdict is that this is a new instance of a known failure mode. That framing is still individually actionable: the general risk around piecewise domains and simplification may be broadly understood, but this exact reproducible case exposes a concrete regression-test target for core algebraic identities and relational simplification.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, Piecewise, S, symbols

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_incomplete_piecewise_undefined_branch_survives_core_identities(a):
    x = symbols("x", real=True)
    p = Piecewise((a + 1, x > a))
    bad_point = a - 1

    assert p.subs(x, bad_point) is S.NaN
    assert (p * 0).subs(x, bad_point) is S.NaN
    assert (p - p).subs(x, bad_point) is S.NaN
    assert Eq(p, p).subs(x, bad_point) is not S.true
```